DESIGN AND ANALYSIS OF ALGORITHMS

LAB WORKBOOK WEEK – 8

NAME: Kurra Venkata Gokul

ROLL NUMBER: CH.SC.U4CSE24121

CLASS: CSE-B

Huffman Coding:

DATA ANALYTICS AND INTELLIGENCE LABORATORY

Code:

```c
//CH.SC.U4CSE24121
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define SIZE 100
struct HuffNode {
    char symbol;
    int weight;
    struct HuffNode *lchild, *rchild;
};
struct HuffNode* newHuffNode(char symbol, int weight) {
    struct HuffNode* tempNode =
        (struct HuffNode*)malloc(sizeof(struct HuffNode));
    tempNode->symbol = symbol;
    tempNode->weight = weight;
    tempNode->lchild = tempNode->rchild = NULL;
    return tempNode;
}
void arrange(struct HuffNode* list[], int count) {
    for(int x = 0; x < count-1; x++) {
        for(int y = x+1; y < count; y++) {
            if(list[x]->weight > list[y]->weight) {
                struct HuffNode* swapNode = list[x];
                list[x] = list[y];
                list[y] = swapNode;
            }
        }
    }
}
void displayCodes(struct HuffNode* head, int path[], int level,
                  int *compressedBits, int *charCount) {
    if(head->lchild) {
        path[level] = 0;
        displayCodes(head->lchild, path, level+1,
                     compressedBits, charCount);
    }
```

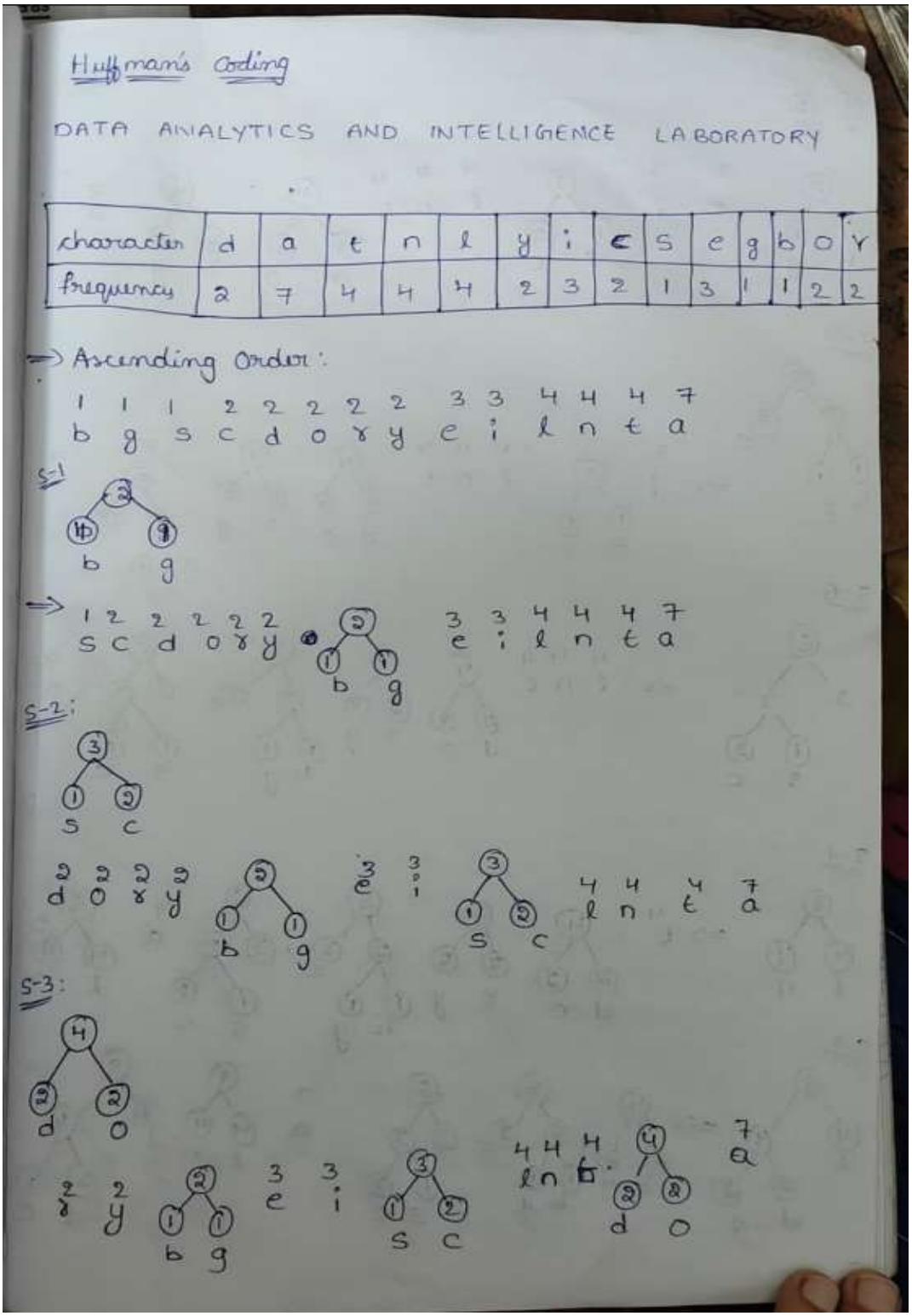
```
    if(head->rchild) {
        path[level] = 1;
        displayCodes(head->rchild, path, level+1,
                     compressedBits, charCount);
    }
    if(!head->lchild && !head->rchild) {
        printf("%c : ", head->symbol);
        for(int i = 0; i < level; i++)
            printf("%d", path[i]);
        printf("  (freq=%d, length=%d)\n",
               head->weight, level);
        *compressedBits += head->weight * level;
        *charCount += head->weight;
    }
}
int main() {
    char inputStr[] =
    "DATA ANALYTICS AND INTELLIGENCE LABORATORY";
    int frequency[256] = {0};
    for(int i = 0; inputStr[i] != '\0'; i++) {
        if(inputStr[i] != ' ')
            frequency[(int)inputStr[i]]++;
    }
    struct HuffNode* nodeList[SIZE];
    int nodeTotal = 0;
    for(int j = 0; j < 256; j++) {
        if(frequency[j] > 0) {
            nodeList[nodeTotal++] =
                newHuffNode((char)j, frequency[j]);
        }
    }
```

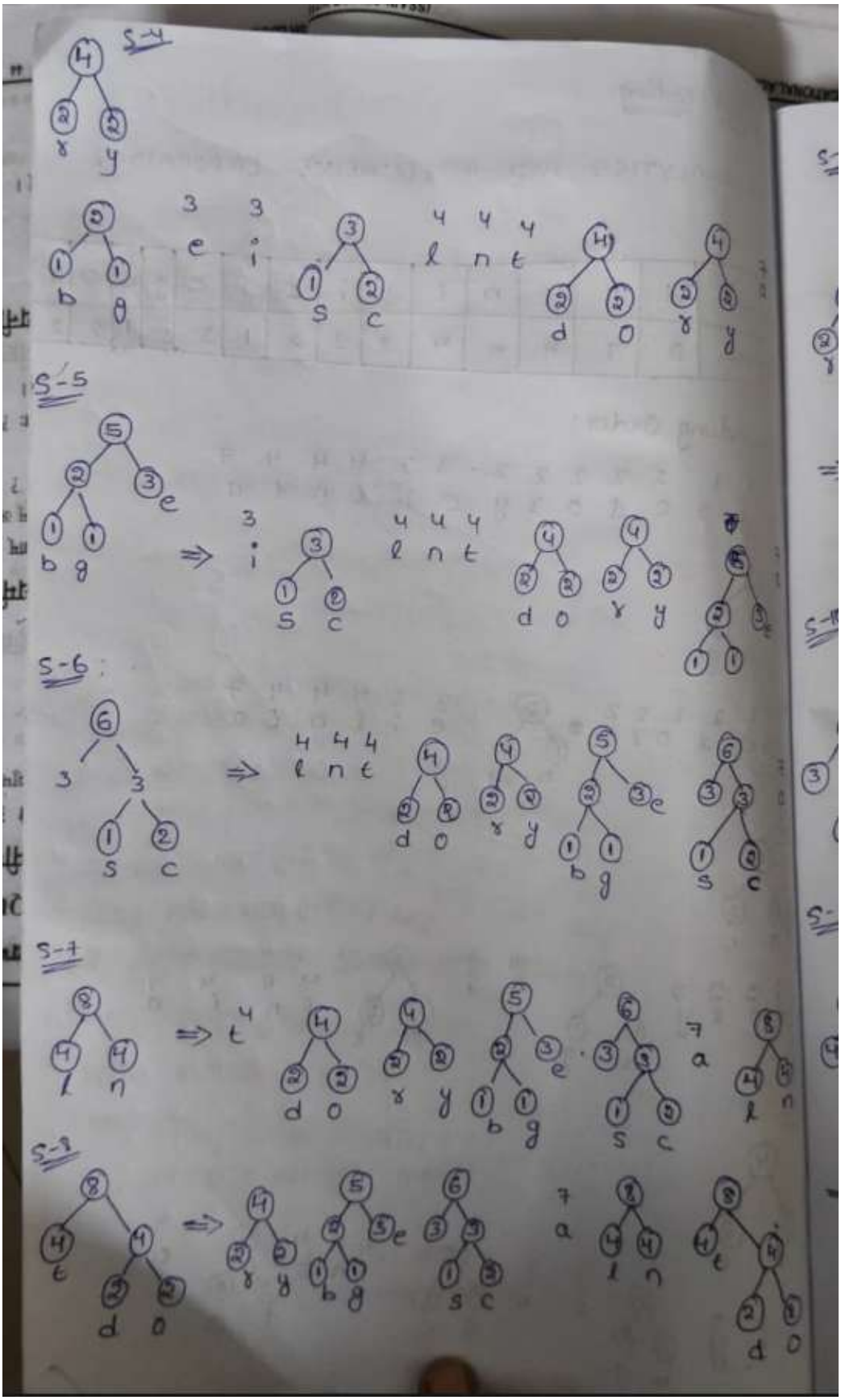
```
    while(nodeTotal > 1) {
        arrange(nodeList, nodeTotal);
        struct HuffNode* firstNode = nodeList[0];
        struct HuffNode* secondNode = nodeList[1];
        struct HuffNode* mergedNode =
            newHuffNode('#',
                        firstNode->weight +
                        secondNode->weight);
        mergedNode->lchild = firstNode;
        mergedNode->rchild = secondNode;
        nodeList[0] = mergedNode;
        nodeList[1] = nodeList[nodeTotal-1];
        nodeTotal--;
    }
    struct HuffNode* treeRoot = nodeList[0];
    int codeTrack[SIZE];
    int totalCompressed = 0;
    int totalCharacters = 0;
    printf("Huffman Codes:\n\n");
    displayCodes(treeRoot, codeTrack, 0,
                 &totalCompressed, &totalCharacters);
    printf("\nTotal Compressed Bits = %d\n",
           totalCompressed);
    float averageLength =
        (float)totalCompressed / totalCharacters;
    printf("Average Code Length = %.2f bits\n",
           averageLength);
    return 0;
}
```

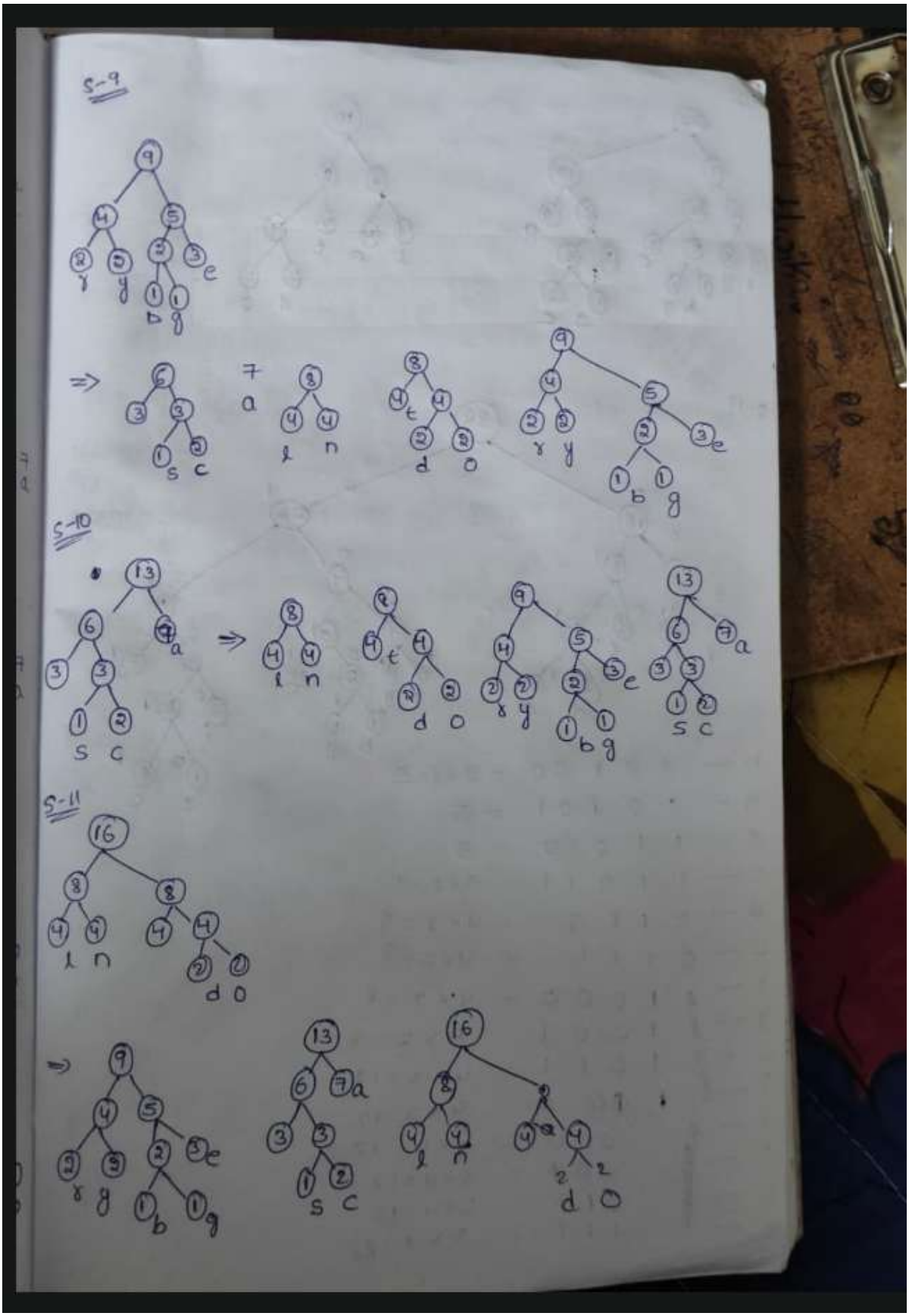
Output:

```
Huffman Codes:

R : 0000  (freq=2, length=4)
D : 0001  (freq=2, length=4)
C : 0010  (freq=2, length=4)
O : 0011  (freq=2, length=4)
L : 010  (freq=4, length=3)
T : 011  (freq=4, length=3)
N : 100  (freq=4, length=3)
Y : 1010  (freq=2, length=4)
S : 10110  (freq=1, length=5)
B : 101110  (freq=1, length=6)
G : 101111  (freq=1, length=6)
E : 1100  (freq=3, length=4)
I : 1101  (freq=3, length=4)
A : 111  (freq=7, length=3)

Total Compressed Bits = 138
Average Code Length = 3.63 bits
```

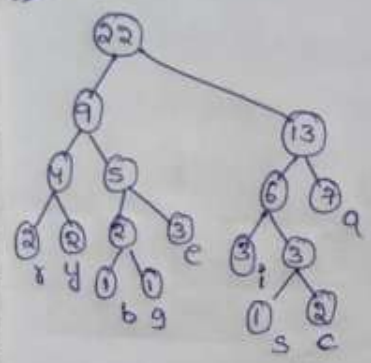
Working:

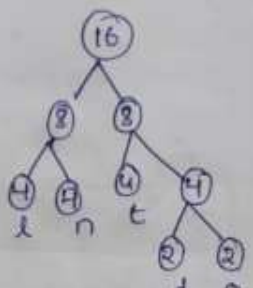

Huffman's coding
DATA ANALYTICS AND INTELLIGENCE LABORATORY
character d a t n l y i € s e g b o r
frequency 2 7 4 4 4 2 3 2 1 3 1 1 2 2
⇒ Ascending order:
1 1 1 2 2 2 2 2 3 3 4 4 4 7
b g s c d o r y e i l n t a
S-1
S-2:
S-3:


S-4
S-5
S-6
S-7
S-8

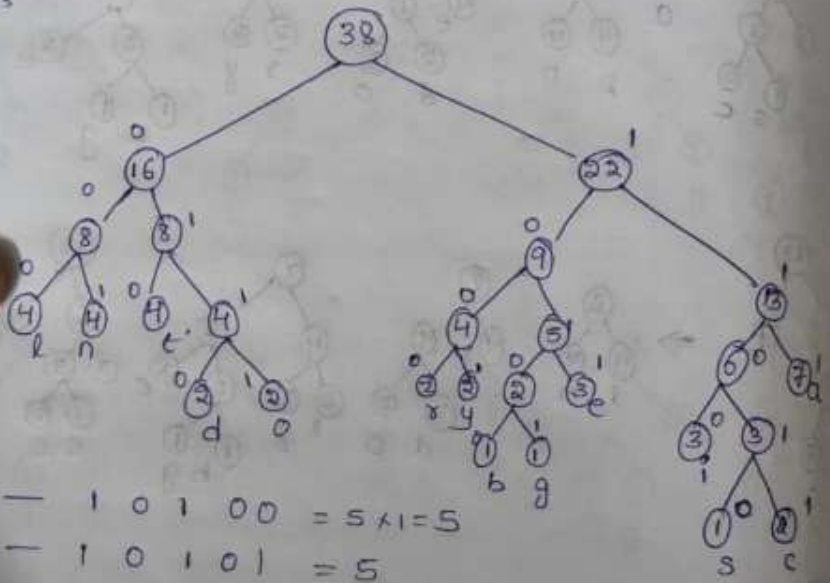

S-9
S-10
S-11

S-12





S-13



b — 1 0 1 00 = 5×1=5

g — 1 0 1 0 1 = 5

s — 1 1 0 1 0 = 5

c — 1 1 0 1 1 = 5×2=10

d — 0 1 1 0 = 4×2=8

o — 0 1 1 1 = 4×2=8

γ — 1 0 0 0 = 4×2=8

y — 1 0 0 1 = 4×2=8

e — 1 0 1 1 = 4×3=12

i — 1 1 0 0 = 4×3=12

l — 0 0 0 = 3×4=12

n — 0 0 1 = 3×4=12

t — 0 1 0 = 3×4=12

a — 1 1 1 = 3×7=21

Time Complexity:

The program repeatedly arranges (sorts) the nodes in ascending order and merges the two smallest nodes until only one node remains.
Since simple nested loops are used for sorting and sorting happens inside a loop, the total time increases significantly.

- Best / Average Case = $O(n^3)$
- Worst Case = $O(n^3)$

Space Complexity:

Space is used for storing the frequency array, node list, and the Huffman tree. Recursion is used to generate Huffman codes, and recursion depth depends on tree height.

- Average Case = $O(n)$
- Worst Case = $O(n)$